

CENTRO UNIVERSITÁRIO MAURÍCIO DE NASSAU DE CACOAL
UNINASSAU

Análise e Desenvolvimento de Sistemas

ANA LUIZA MARCELINO DE CARLI
EMILY PEREIRA GONÇALVES
KEROLLAYNE AKEMY GONÇALVES PEREIRA

ADOTAPET

Arquitetura e documentação da API de adoção de animais

Cacoal – RO

2026

ANA LUIZA MARCELINO DE CARLI
EMILY PEREIRA GONÇALVES
KEROLLAYNE AKEMY GONÇALVES PEREIRA

ADOTAPET

Arquitetura e documentação da API de adoção de animais

Trabalho apresentado à disciplina de
Arquitetura de Software e Computação em
Nuvem, do Centro Universitário Maurício de
Nassau de Cacoal, como parte da avaliação N1
da Atividade Prática 2026.

Professor: Elemar Leonel Balduino Oliveira

Cacoal – RO

2026

RESUMO

Este trabalho documenta a arquitetura atual do AdotaPet, uma API local para cadastro de animais e gerenciamento de solicitações de adoção. A aplicação utiliza NestJS 11, TypeScript e Node.js, com organização em módulos de autenticação, animais, adoções e saúde. Controllers recebem requisições HTTP, DTOs validam os dados e serviços executam regras de negócio. O StoreService mantém mapas em memória e grava usuários, animais e solicitações em um arquivo JSON local, recuperado a cada inicialização. As sessões permanecem temporárias. A autenticação utiliza tokens opacos de sessão, enquanto a autorização verifica o perfil e a responsabilidade pelo animal. O documento apresenta as visões C4 de contexto, containers e componentes, a modelagem UML correspondente às interfaces do código e a sequência de aprovação de uma solicitação. Também descreve o contrato OpenAPI, os exemplos disponíveis no Swagger e a verificação por testes HTTP. A análise descreve a recuperação dos dados após reinício e os limites de operação em um único processo. A implantação com Docker em uma VPS e os requisitos adicionais de produção são apresentados como evolução prevista para a entrega final de novembro. A documentação oferece uma representação verificável da implementação utilizada na apresentação da N1 de 1 de outubro de 2026.

Palavras-chave: arquitetura de software; API; adoção de animais; NestJS; modelo C4.

SUMÁRIO

1 INTRODUÇÃO.....	4
1.1 Requisitos da N1.....	4
2 ARQUITETURA E FERRAMENTAS	5
2.1 Organização do código.....	6
3 VISÕES ARQUITETURAIS C4	7
3.1 Contexto do sistema	7
3.2 Containers	8
3.3 Componentes.....	9
4 MODELAGEM DE DADOS.....	10
4.1 Relações e integridade.....	11
4.2 Persistência local.....	12
5 COMPORTAMENTO E SEGURANÇA	13
5.1 Autenticação e autorização	13
5.2 Validação e respostas de erro	13
5.3 Aprovação de adoção	14
6 CONTRATO HTTP E SWAGGER.....	15
6.1 Demonstração no Swagger.....	16
7 VERIFICAÇÃO DA IMPLEMENTAÇÃO.....	17
8 EVOLUÇÃO PREVISTA PARA NOVEMBRO	18

1 INTRODUÇÃO

O AdotaPet é uma API para conectar o cadastro de animais ao processo de solicitação e decisão de adoção. A versão documentada executa localmente em um único processo Node.js. O objetivo desta entrega é demonstrar uma arquitetura definida, apresentar as visões C4 e a UML e evidenciar a correspondência entre o modelo e o código desenvolvido.

O escopo funcional inclui cadastro público de usuários, login, consulta e cadastro de animais, criação de solicitações e aprovação ou recusa pelo responsável. Não há interface de produto com páginas de navegação; o Swagger é a ferramenta de documentação e experimentação das rotas HTTP. A aplicação responde em JSON.

Esta revisão descreve a implementação disponível para a N1 de 01/10/2026. PostgreSQL, Redis, JWT e Docker não estão presentes na execução atual. Docker e implantação em VPS pertencem à entrega final prevista para novembro. As evoluções estão separadas na seção 8.

1.1 Requisitos da N1

Quadro 1 — Correspondência com a entrega da N1

Requisito	Evidência no projeto
Arquitetura definida e ferramentas	Monólito modular NestJS; organização e ferramentas na seção 2.
C4 de Containers e Componentes	Figuras 3 e 4 representam a execução atual.
Modelagem de classes UML	Figura 5 corresponde às interfaces de domain/entities.ts.
Código baseado na modelagem	Serviços usam User, Animal e Application; validação na seção 7.

Fonte: elaboração própria (2026).

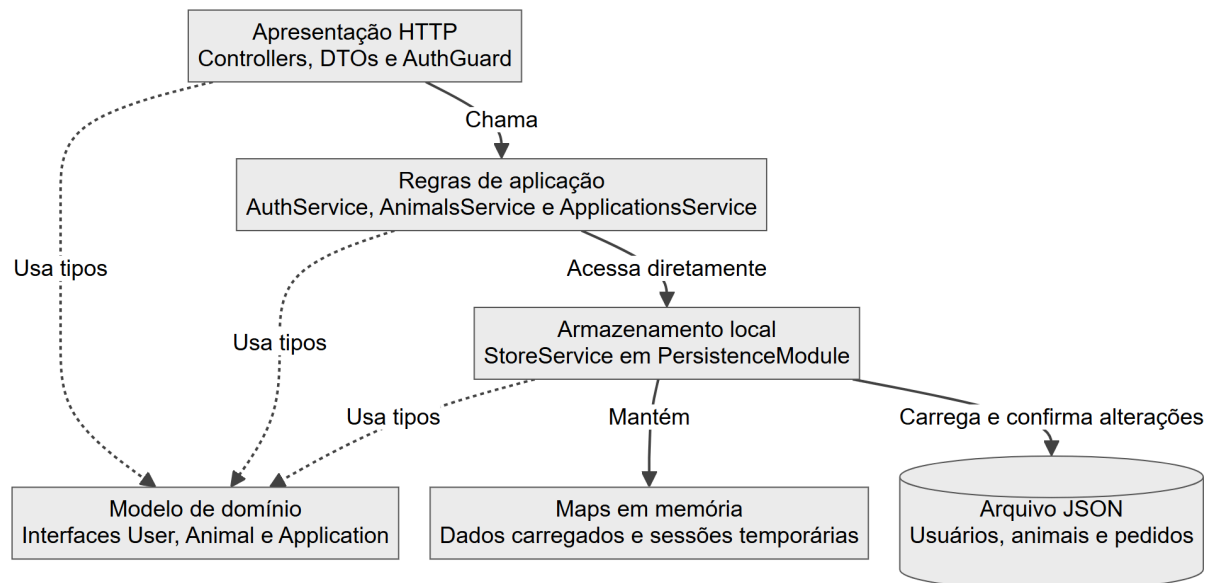
A versão local preserva usuários, animais e solicitações no arquivo data/adotapet.json. A inicialização recupera os registros existentes, sem recriar seus identificadores. As sessões permanecem em memória, exigindo novo login após reiniciar.

2 ARQUITETURA E FERRAMENTAS

A arquitetura atual é um monólito modular com separação de responsabilidades. AuthModule, AnimalsModule, ApplicationsModule e HealthModule compõem a mesma aplicação. PersistenceModule fornece um StoreService compartilhado. A API utiliza o adaptador Express do NestJS e não depende de serviços externos para executar.

Controllers tratam a entrada HTTP, DTOs definem validação e documentação, services aplicam as regras de negócio e as interfaces de domínio descrevem os dados. Os serviços acessam diretamente o StoreService. Portanto, a implementação ainda não possui repositórios abstratos nem a inversão de dependência de uma Clean Architecture completa.

Figura 1 — Separação de responsabilidades da implementação atual



Fonte: elaboração própria (2026).

Quadro 2 — Ferramentas efetivamente utilizadas

Ferramenta	Uso
TypeScript e Node.js	Linguagem e ambiente de execução.
NestJS 11 e Express	Módulos, injeção de dependências e rotas HTTP.
class-validator e class-transformer	Validação dos DTOs e transformação de entradas.
@nestjs/swagger	Documentação OpenAPI em /docs e /docs-json.
Jest e @nestjs/testing	Testes HTTP com instância isolada da aplicação.
Mermaid	Fontes editáveis dos diagramas.

Fonte: elaboração própria (2026).

2.1 Organização do código

Quadro 3 — Pastas e responsabilidades

Local	Responsabilidade
src/modules/auth/	Controllers, serviços, guards, DTOs e AuthModule.
src/modules/animals/	Controllers, serviços, DTOs e AnimalsModule.
src/modules/applications/	Controllers, serviços, DTOs e ApplicationsModule.
src/modules/health/	HealthController e HealthModule.
src/domain/entities.ts	Interfaces User, Animal, Application e tipos de perfil e estado.
src/infrastructure/persistence/	StoreService, validação do snapshot e PersistenceModule.
src/common/	Decorador Trim, funções de senha e metadados Swagger.
src/app.module.ts	Importação dos módulos da aplicação.
src/setup-app.ts e src/main.ts	Configuração HTTP, Swagger e inicialização.
test/	Testes de contrato HTTP e persistência local.

Fonte: elaboração própria (2026).

Cada módulo registra seus próprios controllers e providers. AuthModule exporta AuthService e AuthGuard; AnimalsModule exporta AnimalsService para uso nas adoções. PersistenceModule exporta o StoreService, mantendo o estado compartilhado no contexto da aplicação.

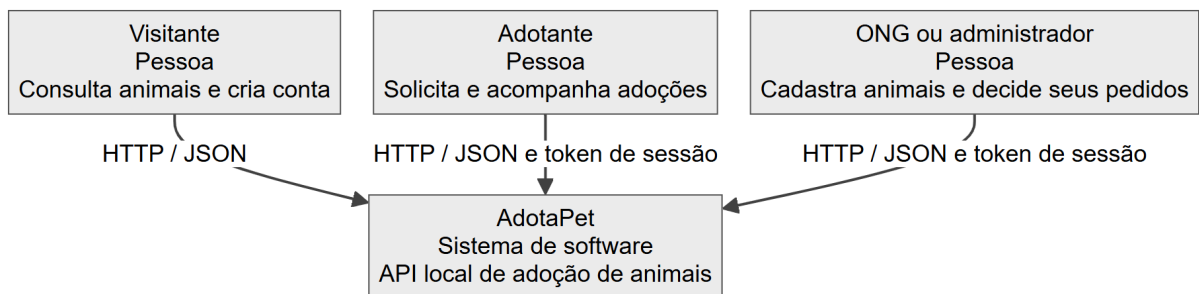
A separação facilita localizar responsabilidades e desenvolver funcionalidades em equipe. Como os serviços ainda dependem da implementação concreta de armazenamento, uma futura troca de persistência exigirá refatoração. Criar pastas, isoladamente, não implementa contratos de repositório nem transações de banco.

3 VISÕES ARQUITETURAIS C4

O modelo C4 descreve a arquitetura em diferentes níveis de detalhe. A visão de Containers identifica aplicações e armazenamentos; a de Componentes detalha as responsabilidades internas de um container (Brown, [s.d.]a; Brown, [s.d.]b). As figuras desta seção retratam o código atual.

3.1 Contexto do sistema

Figura 2 — Contexto atual do AdotaPet

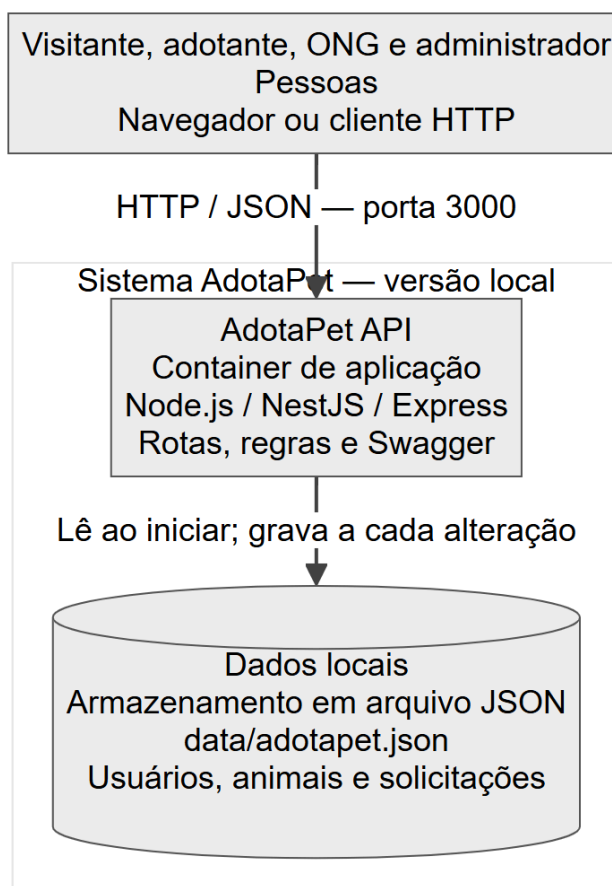


Fonte: elaboração própria (2026).

Visitantes consultam animais e criam contas. Adotantes solicitam adoção e consultam seus pedidos. ONGs e administradores cadastram animais e decidem pedidos apenas dos animais sob sua responsabilidade. O perfil ADMIN não permite aprovar solicitações de qualquer ONG.

3.2 Containers

Figura 3 — Containers da aplicação e do armazenamento local



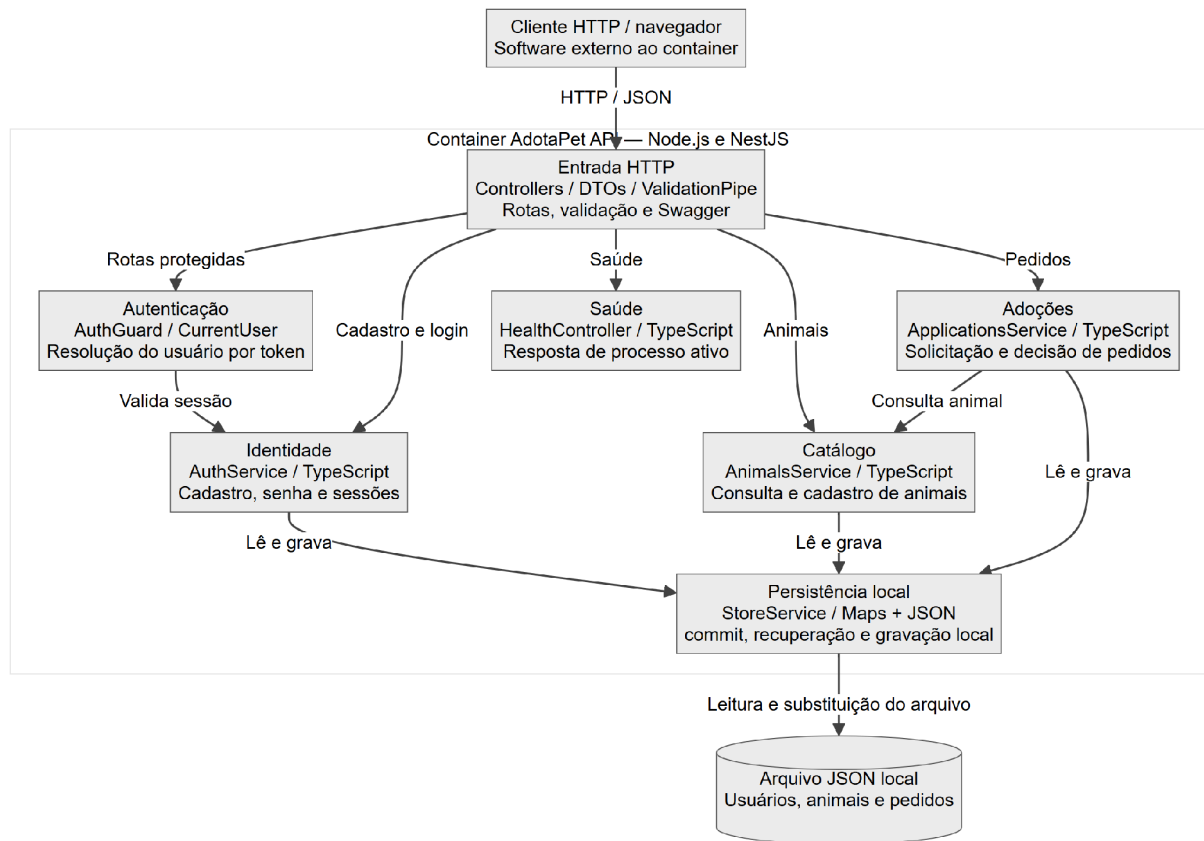
Fonte: elaboração própria (2026).

A visão C4 distingue a API Node.js/NestJS e seu armazenamento persistente em arquivo JSON. O arquivo reside no mesmo computador e não é um servidor de banco de dados. O Swagger UI é servido pela API; StoreService e os mapas são componentes internos do processo.

As requisições locais utilizam HTTP na porta 3000 por padrão. As rotas de negócio usam o prefixo /api, e a documentação está em /docs. Não existem proxy Nginx, conexão PostgreSQL ou conexão Redis nesta versão. Container C4 representa uma unidade arquitetural e não exige uso de Docker.

3.3 Componentes

Figura 4 — Componentes internos da API atual



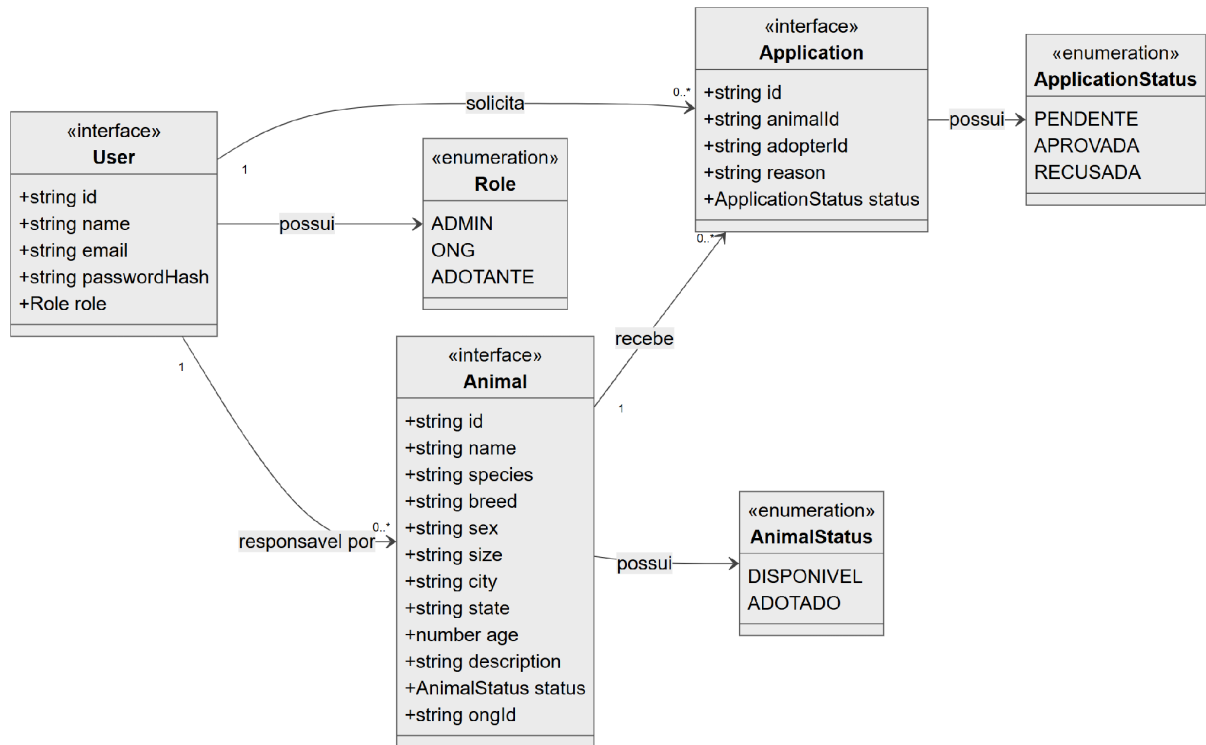
Fonte: elaboração própria (2026).

A entrada HTTP reúne os controllers, DTOs e ValidationPipe. O AuthGuard consulta AuthService para resolver o usuário da sessão. AuthService, AnimalsService e ApplicationsService acessam StoreService; ApplicationsService também consulta AnimalsService. StoreService carrega e valida o arquivo na inicialização e confirma alterações pelo método commit. HealthController informa que o processo está ativo.

As setas indicam colaboração, não a ordem exata do ciclo HTTP. Nas rotas protegidas, o guard executa antes da validação dos parâmetros e da chamada ao controller. AppModule e os módulos funcionais registram as dependências; não são processos separados.

4 MODELAGEM DE DADOS

Figura 5 — Interfaces de domínio e associações da implementação



Fonte: elaboração própria (2026).

`User`, `Animal` e `Application` são interfaces TypeScript em `src/domain/entities.ts`, representadas na UML com esse estereótipo. Identificadores são strings contendo UUIDs; `age` é `number` no código e é validado como inteiro não negativo no DTO de criação. Não há atributos de criação ou atualização temporal nas entidades atuais.

`Role` e `AnimalStatus` são uniões de literais TypeScript. `ApplicationStatus` é um nome conceitual do diagrama para a união inline do campo `status` de `Application`, não um tipo exportado no código. As enumerações UML apresentam os valores permitidos, sem afirmar que existam enums em tempo de execução.

`passwordHash` consta na interface interna de `User`. `PublicUser` omite esse campo, e `AuthService` constrói explicitamente a resposta pública. Por isso o hash não aparece nas respostas de cadastro e login.

4.1 Relações e integridade

Quadro 4 — Correspondência entre modelo e implementação

Elemento	Correspondência
User → Animal	Animal.ongId referencia o usuário responsável, com papel ONG ou ADMIN.
User → Application	Application.adopterId identifica o ADOTANTE que criou o pedido.
Animal → Application	Application.animalId identifica o animal solicitado.
Multiplicidades	Um usuário pode ter vários animais ou pedidos; cada pedido pertence a um animal e adotante.
Estados de Animal	DISPONIVEL e ADOTADO.
Estados de Application	PENDENTE, APROVADA e RECUSADA.

Fonte: elaboração própria (2026).

As associações são garantidas pelas chamadas dos serviços, não por chaves estrangeiras de um banco. Os dados carregados ficam em Maps indexados por UUID. O arquivo registra as coleções users, animals e applications e a versão do formato. As contas locais e a cadela Mel são criadas somente quando o arquivo ainda não existe.

A criação de solicitação exige animal disponível e impede outro pedido pendente do mesmo adotante para o mesmo animal. A decisão exige pedido pendente e responsável autenticado. Uma aprovação altera o animal para ADOTADO e recusa os outros pedidos ainda pendentes.

A sessão é uma estrutura técnica separada: o mapa sessions associa um token aos campos userId e expiresAt. Ela não é uma entidade do domínio de adoção e não aparece como tabela persistente. As sessões desaparecem ao encerrar o processo; os dados de negócio permanecem no arquivo. A senha é salva somente como hash, e os tokens de sessão não são gravados.

4.2 Persistência local

O caminho padrão é `data/adotapet.json`, relativo à raiz do projeto. `DATA_FILE` permite escolher outro caminho pelo ambiente do processo. A aplicação deve ser iniciada sempre na mesma pasta ou com um caminho absoluto configurado. A pasta `data` é ignorada pelo Git e deve ser preservada ao atualizar o código.

Cada cadastro e decisão executa `commit`, que captura o estado anterior, aplica a alteração, escreve o snapshot completo em um arquivo temporário no mesmo diretório, sincroniza a escrita e renomeia o temporário sobre o arquivo definitivo. A gravação ocorre antes do retorno HTTP de sucesso e não depende do encerramento normal da API.

Se a gravação falhar, o estado em memória é restaurado e a requisição recebe 503. Se um arquivo existente estiver malformado ou violar as validações estruturais e de relacionamentos, a inicialização falha sem substituir seu conteúdo. Deve-se corrigir o arquivo ou restaurar um backup; não há recuperação automática de arquivos corrompidos.

O arquivo deve ser usado por apenas um processo da API. Não há bloqueio entre processos; duas instâncias escrevendo no mesmo caminho podem perder alterações. A gravação síncrona do arquivo inteiro é adequada ao volume pequeno da demonstração, mas limita o desempenho. A substituição do arquivo não elimina a necessidade de backup nem garante proteção contra todas as falhas de hardware.

Para verificar a persistência, cadastrar um animal e uma solicitação, parar a API, iniciar novamente e fazer novo login. Os registros e seus identificadores devem continuar iguais. O mesmo procedimento deve preservar uma aprovação e as recusas associadas. A execução de testes utiliza arquivos temporários e não a pasta `data` real.

5 COMPORTAMENTO E SEGURANÇA

5.1 Autenticação e autorização

O cadastro público aceita ONG e ADOTANTE, com ADOTANTE como padrão. E-mails são normalizados para minúsculas e duplicatas retornam 409. Senhas são armazenadas como hash scrypt com salt individual; o código atual utiliza processamento síncrono.

O login cria um token aleatório de 32 bytes, representado por 64 caracteres hexadecimais. O token é opaco, não JWT, e identifica uma sessão em memória válida por 24 horas. O AuthGuard exige Authorization: Bearer <token> e consulta a sessão em cada acesso protegido.

Quadro 5 — Permissões atuais

Operação	Permissão
Consultar animais e saúde	Pública.
Cadastrar usuário e fazer login	Pública, com validação de entrada.
Cadastrar animal	ONG ou ADMIN.
Solicitar adoção	Somente ADOTANTE.
Listar solicitações	ADOTANTE: próprias; ONG/ADMIN: dos seus animais.
Aprovar ou recusar pedido	ONG ou ADMIN responsável pelo animal.

Fonte: elaboração própria (2026).

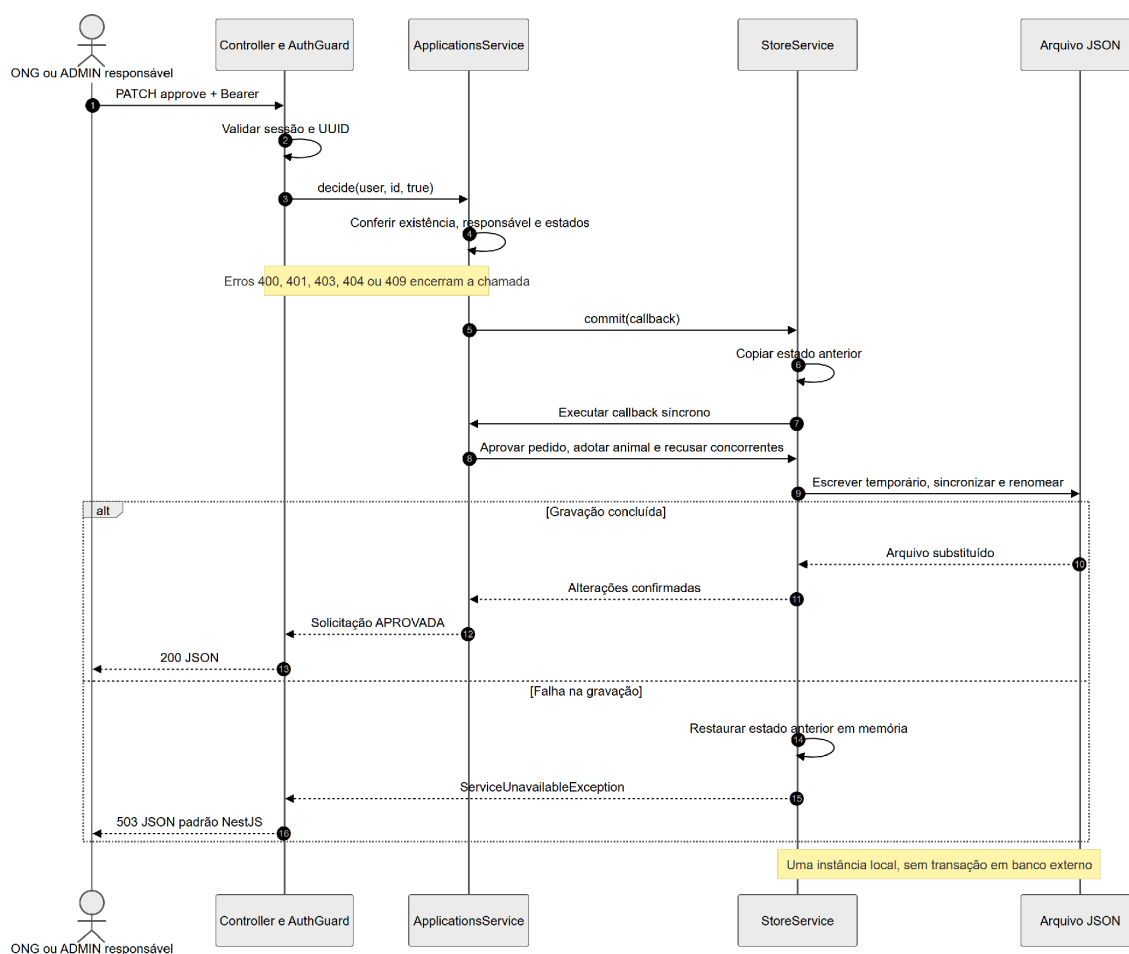
5.2 Validação e respostas de erro

ValidationPipe transforma a entrada, aplica os validadores e rejeita propriedades não previstas nos DTOs. ParseUUIDPipe valida os identificadores de rota. O formato atual de erro é o padrão NestJS: statusCode, message e error. message pode ser texto ou uma lista de mensagens de validação.

A API usa 400 para entrada inválida, 401 para autenticação ausente ou inválida, 403 para falta de permissão, 404 para recurso inexistente, 409 para conflitos de negócio e 503 se não for possível salvar uma alteração. Ainda não há filtro global Problem Details, rate limiting ou logs JSON com Request-ID.

5.3 Aprovação de adoção

Figura 6 — Sequência de aprovação com persistência local



Fonte: elaboração própria (2026).

ApplicationsService.decide valida a decisão e chama StoreService.commit. O callback altera o pedido, o animal e os demais pendentes como uma única operação local. A resposta de sucesso ocorre após a substituição do arquivo. Uma falha de gravação restaura os mapas anteriores e retorna 503. A solução não fornece transações PostgreSQL nem coordenação entre processos.

6 CONTRATO HTTP E SWAGGER

O contrato OpenAPI é gerado a partir dos metadados dos controllers, dos DTOs e dos decoradores de documentação compartilhados. O Swagger UI está em <http://localhost:3000/docs> e o JSON em [/docs-json](#). A integração permite registrar descrições, parâmetros e respostas das operações (NestJS, [s.d.]).

Quadro 6 — Rotas existentes e status de sucesso

Método e rota	Sucesso
POST /api/auth/register	201
POST /api/auth/login	201
GET /api/animals	200
GET /api/animals/{id}	200
POST /api/animals	201
POST /api/animals/{animalId}/applications	201
GET /api/applications/me	200
PATCH /api/applications/{id}/approve	200
PATCH /api/applications/{id}/reject	200
GET /api/health	200

Fonte: elaboração própria (2026).

A listagem de animais permite filtros por species, city e status, sem paginação. O filtro de cidade ignora diferenças entre maiúsculas e minúsculas; species usa comparação exata. GET /api não possui uma rota e retorna 404. O endpoint de saúde informa o processo ativo, sem verificar bancos externos.

Os exemplos de resposta apresentam a estrutura de usuário público, animal, solicitação e login. As rotas documentam os erros previstos e o esquema Bearer identifica corretamente o token opaco. Os UUIDs e tokens exemplificados não são credenciais válidas nem registros garantidos na execução.

6.1 Demonstração no Swagger

Primeiro, executar `npm ci` e `npm run start:dev`. A aplicação usa a porta 3000 por padrão; PORT é lida do ambiente do processo. O arquivo `.env` não é carregado automaticamente. As contas locais `ong@adotapet.local`, `adotante@adotapet.local` e `admin@adotapet.local` usam a senha de demonstração 123456.

Fazer login como adotante e copiar o `accessToken` real retornado. Em Authorize, colar somente esse valor. Consultar `GET /api/animals` e copiar o id de um animal disponível sem aspas. Esse identificador deve preencher `animalId` na criação de solicitação.

Enviar um motivo de adoção, ou um objeto vazio quando não houver motivo. A resposta 201 traz o id da solicitação e o estado `PENDENTE`. Guardar esse id, pois as rotas de decisão recebem o identificador da solicitação, não o do animal.

Fazer login como a ONG responsável e substituir o token no Authorize. Consultar os pedidos em `GET /api/applications/me`. Executar a aprovação ou recusa e verificar o estado retornado. Após aprovação, consultar o animal para confirmar `ADOTADO`. Uma segunda decisão sobre o mesmo pedido retorna 409.

Os exemplos preenchidos nos DTOs ajudam a iniciar os testes, mas o cadastro de usuário exige um e-mail ainda não utilizado. Os exemplos de respostas de erro são representativos: o texto específico depende da validação ou regra que rejeitou a requisição.

7 VERIFICAÇÃO DA IMPLEMENTAÇÃO

A verificação utiliza `npm run typecheck`, `npm run build` e `npm test -- --runInBand`. A suíte reúne oito testes do contrato e fluxo HTTP e três testes de persistência. Os testes funcionais usam um `StoreService` isolado em memória; os testes de persistência usam arquivos temporários próprios. Nenhum teste altera o arquivo de dados da execução normal.

Quadro 7 — Cobertura dos testes existentes

Grupo	Comportamento verificado
Saúde e consulta	Saúde, filtros, status inválido, UUID inválido e animal inexistente.
Cadastro e login	Normalização de e-mail, duplicidade, perfil proibido, senha e login.
Sessões	Ausência de autenticação, token inválido, malformado e expirado.
Animais	Permissões, campos controlados pelo servidor e validação.
Aprovação	Pedido duplicado, aprovação única e recusa dos concorrentes.
Responsabilidade	Outra ONG ou adotante não decide; recusa preserva disponibilidade.
Administrador	Decisão de pedido de animal cadastrado pelo próprio ADMIN.
OpenAPI	Schemas, descrições, exemplos, respostas e autenticação das dez operações.

Fonte: elaboração própria (2026).

Na revisão de 24/09/2026, os testes de contrato verificam respostas de sucesso e erro. Os testes de persistência recriam a aplicação para conferir usuários, animais, pedidos e aprovação, verificam a rejeição de arquivo inválido e simulam falha de gravação. Os testes de contrato foram ampliados para conferir os códigos de resposta documentados e comparar a estrutura de exemplos de login e erro com respostas reais. A documentação de sucesso não inclui `passwordHash`.

Esses testes verificam a aplicação local. Comprovam a recuperação após recriar a aplicação, mas não medem disponibilidade em VPS, desempenho sob carga ou concorrência entre processos. O atendimento dessas características dependerá da evolução prevista para a entrega final.

8 EVOLUÇÃO PREVISTA PARA NOVEMBRO

A entrega final deverá ampliar a versão local conforme os requisitos da disciplina. Esta seção é planejamento e não representa funcionalidades já implementadas. Docker e implantação em VPS estão previstos para novembro, conforme o cronograma informado pelo grupo.

Quadro 8 — Estado atual e evolução prevista

Aspecto	Atual	Previsto
Armazenamento	Arquivo JSON local e Maps carregados.	PostgreSQL com migrações e persistência.
Autenticação	Token opaco e sessão local.	JWT com validação e políticas de acesso.
Controle de requisições	Sem limitador.	Redis e rate limiting.
Criações críticas	Validações de negócio.	Idempotency-Key e registros duráveis.
Aprovação	Commit em arquivo com restauração em caso de falha.	Transação e proteção contra concorrência.
Erros e observabilidade	Erros padrão NestJS.	Problem Details e logs JSON com Request-ID.
Execução	Processo local Node.js.	Docker Compose com API e dois bancos; VPS.

Fonte: elaboração própria (2026).

O modelo relacional e o diagrama de implantação futura são mantidos entre os arquivos Mermaid complementares, identificados como proposta. Eles não substituem os diagramas C4 atuais desta entrega. As escolhas devem ser revalidadas e documentadas quando forem implementadas.

A prioridade da N1 é demonstrar a correspondência entre arquitetura, modelagem e código. A arquitetura atual e seus limites estão explicitados neste relatório, permitindo apresentar o fluxo funcional sem atribuir ao protótipo recursos de produção ainda ausentes.